Institut Supérieur des Mathématiques Appliqueés et d'Informatique de Kairouan

Projet de Fin D'Etudes

Pour obtenir le

Diplôme de Licence Appliquée en Réseaux Informatiques: Technologie de Réseaux et Télécommunication

Présenté et soutenu le 04 / 06 / 2021

par

Wael Ghnimi

Toward the conception and realization of an AI-Based system for Penetration Testing

Composition du jury

Président	Mme/Mr	Sihem Cherif
Encadreur	Mme/Mr	Mohamed Nidhal Abdi
Rapporteur	Mme/Mr	Laroussi Merehben

Année Universitaire 2020-2021

BONAFIDE CERTIFICATE

This is a certification that this project report entitled **"Toward the conception and realization of an AI-Based system for penetration testing"** submitted by **Wael Ghnimi**, in completion of his work on this project under the supervision of **Dr. Mohamed Nidhal Abdi** during the period from February 2021 - June 2021, is approved for the project report submitted in partial achievement of the requirements for the award of Applied Bachelor Degree in Computer Networks: Telecommunication Network Technology, Higher Institute of Applied Mathematics and Computer Science, Kairouan, during the academic year 2020-21.

ACKNOWLEDGEMENT

Without the existence and assistance of many people, this project would not have been possible. I would like to express my gratitude to everyone who helped me complete this work, whether directly or indirectly.

First and foremost, I would like to thank our beloved Doctor, **Dr. Mohamed Nidhal Abdi**, for his invaluable assistance in a variety of ways.

I would like to thank everyone in the White Hacking, Machine Learning, and Python communities for their invaluable assistance, motivation, inspiration, and consistent and timely support during this project.

Finally, I want to express my heartfelt gratitude to my family for their unwavering affection, encouragement, and assistance. I am grateful to my father for always being a friend to me. My parents will always be responsible for providing me with the opportunities and experiences that have shaped who I am. They inspired me to pursue my dreams and discover new paths in life. Without them, this journey would not have been possible, and I dedicate this achievement to them.

Abstract

Penetration testing is the practice of modeling a simulated attack using real-world techniques to find vulnerabilities under controlled conditions. The most common strategy is to exploit those weaknesses manually by a penetration tester based on a signed agreement that includes a scope and rules of engagement. All to help organizations enhance security practices. This manual method is becoming increasingly complex, non-standard, and cannot satisfy the speed and frequency needed for large-scale security solutions. To resolve this, we provide an automated penetration testing system that uses asynchronous methods for deep reinforcement learning to build an autonomous pentesting process.

To introduce the strategy of using machine learning with the penetration testing process, there are two stages. First, we use Nmap to collect relevant server data such as OS type, product name, and opened ports in order to use them in the next phases. Second, the implementation of Asynchronous Advantage Actor-Critic (A3C) with the RPC API of Metasploit framework to find out the easiest way to exploit the attack path from the potential candidate.

Keywords : penetration testing, machine learning, reinforcement learning, Asynchronous Advantage Actor-Critic

Contents

Abstract **i**

List of Figures **iv**

List of Tables **v**

1 Synopsis **1**
1.1 Introduction 1
1.2 Objective 1
1.3 Goals 1
1.4 Problem Statement 2
1.5 Project Work Structure 2

2 Related Work **3**
2.1 Automated Penetration Testing 3
2.2 Reinforcement Learning 4
2.3 Deep Reinforcement learning 5

3 Penetration Testing Basics for Machine Learning **6**
3.1 Penetration testing planning 6
3.2 The Penetration Testing Types 7
3.3 The Penetration Testing Stages 8

4 Asynchronous Advantage Actor Critic (A3C) **13**
4.1 Reinforcement learning background 13
4.2 Asynchronous Advantage Actor Critic 15

5 Experimental Results **20**
5.1 The network design 20
5.2 Metasploit 21
5.3 APTML Application 25
5.4 Testing APTML and Results 32

5.5 Conclusion 36

6 Conclusion and Future Work **38**

6.1 Conclusion 38

6.2 Future Work 38

Appendix

A.1 Preparing Penetration Testing Environment 42

A.2 Program listing 44

List of Figures

2.1 Reinforcement learning process 4

3.1 Penetration Testing Methodology 8
3.2 Workflow of the Scanning Phase 10
3.3 Scenario of Pivoting Chain 12

4.1 Reinforcement Learning with Cart-Pole 14
4.2 The Actor-Critic Architecture 17
4.3 Actor-Critic Neural Network in CartPole 17
4.4 Asynchronous Learning 19

5.1 Example network with four subnets 20
5.2 The system component of APTML 26
5.3 Conceptual diagram of APTML 27
5.4 Learning mode: Port Scanning 32
5.5 Learning mode: Exploit process 33
5.6 Learning mode: Production phase 33
5.7 Metasploit Vs. Metasploit with A3C 35

1 VMware Workstation Pro v16 42
2 Kali Linux Distribution 43

List of Tables

5.1 Rewards values of APTML during the experimental process 34
5.2 Rewards values of Metasploit only during the experimental process . . . 34

Listings

5.1 Metasploit framework 21
5.2 Enabling the Metasploit RPC API 22
5.3 Authenticate to Metasploit using RPC API 24
5.4 Creation of MFSconsole using console.create API 25
5.5 Acquisition of OS type 28
5.6 Acquisition of "port number", "protocol", "product name", and "version" 28
5.7 Exploit Module Types 29
5.8 exploit target 29
5.9 payload Lists 30

List of Algorithms

1 ϵ-greedy Q-Learning Algorithm 5

Abbreviations

A3C	Asynchronous Advantage Actor-Critic
API	Application Programming Interface
DMZ	Demilitarized Zone
DQL	Deep Q-learning
DRL	Deep Reinforcement Learning
HTTPS	Hypertext Transfer Protocol Secure
IPS	Intrusion Prevention System
OS	Operating System
PTES	Penetration Testing Execution Standard
QL	Quality learning
RL	Reinforcement learning
RPC	Remote procedure call
ReLU	Rectified Linear Unit
RoE	Rules of Engagement
SSL	Secure Sockets Layer
TD	Temporal-difference
UID	User Identifier
VAS	vulnerability assessment agreement
VM	Virtual Machine

Chapter 1

Synopsis

1.1 Introduction

With the common usage of information technology and the consistent existence of security threats, cybersecurity becomes a progressively dominant challenge. Securing enterprise systems requires more than only firewalls, patch management, and user responsibilities; it calls for regular real-world recognition of what is working and what is not.

Penetration testing (pentesting) is an operation to simulate the attacker's methods to bypass security controls and compromise the organization's networks. It is a form of ethical hacking to deal with threats, vulnerabilities, risks, and exploits.

1.2 Objective

The main objective of this project to automate the penetration testing process using modern machine learning methods. However, this combination builds to test the target's security on the perimeter or internal networks to end with a vulnerability assessment report. This report gives the organization a chance to know their business risk and threat area before evil hackers do.

1.3 Goals

- To automate the penetration testing process.
- To investigate the use of penetration testing in Artificial Intelligence (AI).
- Determine the security of systems on perimeter or internal networks.

1.4 Problem Statement

Generally, pentesting is applied manually, while effective, would not meet the expectation of AI security techniques: non-standard, and cannot satisfy the speed and frequency needed for large-scale security solutions. The alternative, in recent years, people tried to automate the pentesting process using AI techniques. Boddy et al. led the first applications of AI in penetration testing [1]. After many years, the main security model simulated external attacks by creating separate "attack actions" combined with some software vulnerability assessment[2]. Unfortunately, these methods do not qualify as a complete penetration testing framework.

To handle these issues, we present a machine learning method targeted the limits of manual process and exceeding the ability of current automated tools. Reinforcement Learning (RL) provides several algorithms that learn from the interaction with the environment and improving themselves to achieve a specific goal. The recent success of RL methods in games such as AlphaGo or CartPole inspired testing them in these different domains, cybersecurity and penetration testing. The achieved attacking approach should be able to exploit a system in a perimeter or internal networks faster than randomly trying to attack distributed targets.

1.5 Project Work Structure

The remainder of this report is organized in 6 different chapters.

1. *Chapter 2*: Details about the related work regarding the machine learning and penetration testing processes. It also discusses some parts of deep machine learning and previous reinforcement learning applications.
2. *Chapter 3*: Defines the penetration testing process and the techniques in each different phase, including some relevant details about different pentesting types.
3. *Chapter 4*: Presents the A3C method used during the machine learning implementation process.
4. *Chapter 5*: Details our research techniques and methods to solve the previous research questions.
5. *Chapter 6*: Concludes the project work and highlights the potential future work that may be accomplished to further researches in this domain.

Chapter 2

Related Work

This project integrates two common domains: penetration testing (in cybersecurity) and RL (in machine learning). Thus, the following section will introduce the division between knowledge backgrounds and splits them into two subsections.

2.1 Automated Penetration Testing

This subsection describes briefly two papers that were specifically relevant to this project in modeling the pentesting environment. The difficulties handled are similar, and the strategies mentioned offered as good inspiration to model our environment. For each paper, the methods and environment used to solve it are explained along with which elements were included or proceeded into the present work.

Applebaum et al. present the importance of pentesting in the organization's security and the weaknesses of the manual process[3]. Furthermore, The network typology used to simulate the attack is complicated. There are three guests in a simulation, the gray agent (as a normal user), the pentester (as an attacker), and the defender (detection side). In this methodology, the pentester can do the reconnaissance phase, exploitation, post-exploitation, and preventive techniques. The authors evaluate numerous strategies for selecting the action to launch: from arbitrary to fixed-strategy and traditional planning. Efficiency measured in terms of the percentage of stations compromised, credentials gained, and sensitive data hacked. They noticed that the working time is much higher for planners than for direct executors. Moreover, they realized that enhanced connectivity reduces performance for all procedures because of the excess of options[3].

The specific modeling of regular user habit, a variation missed by most other techniques, guides the simulation nearer to reality and is bundled in our model.

Sarraute et al. try to simulate the attack on systems with more complicated topologies: developing asymmetrical connections and clustering them into sub-networks. A defender is not modeled, and the actions of the penetration tester are limited to two forms of scans, a determined list of exploits and the option of stopping. Exploits are represented more in-depth: they can crash the machine (denial of service), and unsuccessful attacks can expose further data about the system.[4]

In this project, we discuss in detail an effective pentesting methodology in Chapter 3. Then, we used to automate the exploitation of all possible vulnerable systems and workstations on the perimeter or internal networks for a given target, all without denial of the system services or breach sensitive data, as will be discussed in Chapter 5.

2.2 Reinforcement Learning

This subsection begins by shortly describing Reinforcement Learning (RL), follows by descriptions of standard algorithms, and finishes with a review of some valuable extensions.

Reinforcement learning is an interactive learning method that maps the environment states to specific actions. In RL, the agent attempt to establish effective ways to render the corresponding action to any environment state, all to improve the cumulative reward progressively. [5]

Figure 2.1 shows the RL interaction process. It contains an *agent* that makes sequences of actions a_t based on a sequence of observation *state* s_t of the *environment* and received collection of *rewards* $r_t \in R$. This flow is repeated until the environment achieves *terminal* state s and the episodes finish. [6]

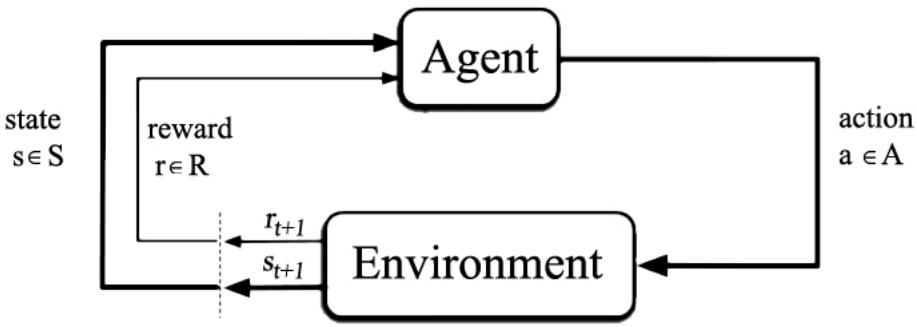

Figure 2.1: Reinforcement learning process

Q-Learning (Quality Learning) is one of the most common RL methods. It is an off-policy algorithm that develops a state-action table $Q(s, a)$ and keeps the utility rates of using action a in state s for all actions attempted in every experienced state[7]. Algorithm 1 describes the pseudo-code of QL functions. On line ❾, the Q value entering is updated regarding the practiced reward in addition to estimated future values. The discount factor $0 < \gamma < 1$, usually near to one and the learning step size (rate) $0 < \alpha < 1$, usually too small and the research probability $0 < \epsilon < 1$. The reduced factor manages how much instant rewards are favored over slowed ones. The low values make more useful behavior when high values contribute to extra visionary behavior. ϵ-greedy, One of the frequent action selection strategies. The agent acts greedily when is prompted to take the following action from state s by exploiting the current data: $argmax_a Q(s, a)$. The strategy my stuck nearest to maximum by operating greedily. To allow the exploration of alternate, potential better paths, an arbitrary action is chosen, with probability ϵ.

Algorithm 1 ϵ-greedy Q-Learning Algorithm

1: **for each** episode **do**
2: $s \leftarrow$ environment's initial state
3: **repeat**
4: **if** exploring (with probability ϵ) **then**
5: $a \leftarrow$ sampled randomly from $\mathcal{A}$
6: **else if** exploiting **then**
7: $a \leftarrow argmax_i Q(s, i)$
8: $r, s' \leftarrow$ environment's reaction to taking action a
9: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma max_i Q(s', i) - Q(s, a)]$
10: $s \leftarrow s'$
11: **until** s is terminal;

The action-state table may be replaced by an approximated model, like a Neural Network. This method is called DQN or Deep Q-learning Network.

2.3 Deep Reinforcement learning

DRL or Deep Reinforcement Learning is a category of methods and strategies that integrate reinforcement learning and deep learning. In DRL, the agent gets state data, chooses excellent actions corresponding to the used strategies, updates the environment's state, then and based on the new environment state it receives the rewards that present the effective actions to the agent [8]. Whenever we use DRL in the pentesting process, the agent takes on the duty of the penetration tester and picks the greatest effective way to find the highest exploits.

Chapter 3

Penetration Testing Basics for Machine Learning

3.1 Penetration testing planning

The main goal of penetration testing is to find all weaknesses that enable the hackers to compromise systems or networks so that a client or organization can realize its business risks and vulnerabilities before the bad guys do. However, to accomplish these goals, pentesters and ethical hackers must establish a hacker mindset.

3.1.1 The Pentester Mindset

A penetration tester need to be pragmatic, adaptable, thinking outside the box. To succeed his mission, the pentester must think differently than other standard network architects and system administrators, working to solve issues in often irregular ways. This process require development hacker mindset, which offers the capability to think as a hacker and be qualified to find tactics to exploit vulnerabilities.

At the same time of using pragmatic methods, the attentiveness and methodology needed to be present along the pentesting process. The process, to be valuable, must be clear and reproducible so that the organization can understand its weaknesses and vulnerabilities and take action to mitigate the risks. [9]

3.1.2 Defining Terms

Threat - is an agent or actor that can cause damage to a specific target or organization's system. Threats include prepared crimes, malware services, and internal employees. Viruses and worms also represent a threat since they could result harm in the system by infecting machines and spread automatically.

Vulnerability - is a weakness in the information system that an attacker can use to end up with damages. Vulnerabilities can exist on several sides in environments, including business processes, deployed software, system configurations, and architectural design.

Risk - Risk is the collaboration of vulnerability and threat. That is, a risk happens when systems have a vulnerability that offers the threat the possibility to attack.

Exploit - Exploit is the method applied by the attacker to use a vulnerability. This exploit can cause a damaging process in the target system. It may be lines of code that return packets of overflow to create a buffer in running software. However, the exploit can be a social engineering attack where employees are led to disclose sensitive information, such as a password or credential data.

3.2 The Penetration Testing Types

There is a large number of penetration test types that can be applied in the process to test systems vulnerabilities based on the assessment agreement.

3.2.1 White Box Testing

The white box approach implemented when the pentester has full access to all the internal network and knows everything about the systems, including the source code. Also, the pentester works in collaboration with the blue team (defensive team) of the organization to find all possible vulnerable assets. This type of test is used when a company needs to discover as much as possible misconfigured machines and vulnerability in its functional system.

3.2.2 Grey Box Testing

The gray box penetration testing is also known as informed penetration testing or assisted black box testing. This method consists to compromise the system with a minimum amount of information on the target and its network. In general, when we use this method, the organization gives an identifier and password. Thus, the given information lets pass-through authentication steps to test the system's confidentiality and integrity.

3.2.3 Black Box Testing

The black box test is a highly used scenario, commonly referred to as penetration testing or ethical hacking. This intrusion test focuses on the externally facing the network's elements or an application without having any prior information. The pentester tried to hack the target from the outside and exploring the system for the first time. Black box testing is performed for a short duration, where true hackers have no time restrictions, which is an important difference comparing to today's Intrusion Prevention System (IPS) technologies.

In this project, we will implement this type of test. Our goal is to evaluate the existent vulnerabilities and simulate real-life hacking scenarios.

3.3 The Penetration Testing Stages

A penetration testing is a continuous cycle of explorations and attacks against a specific target. The assessment should be planned, structured, and calculated. This is how the process of a pentesting needed to be:

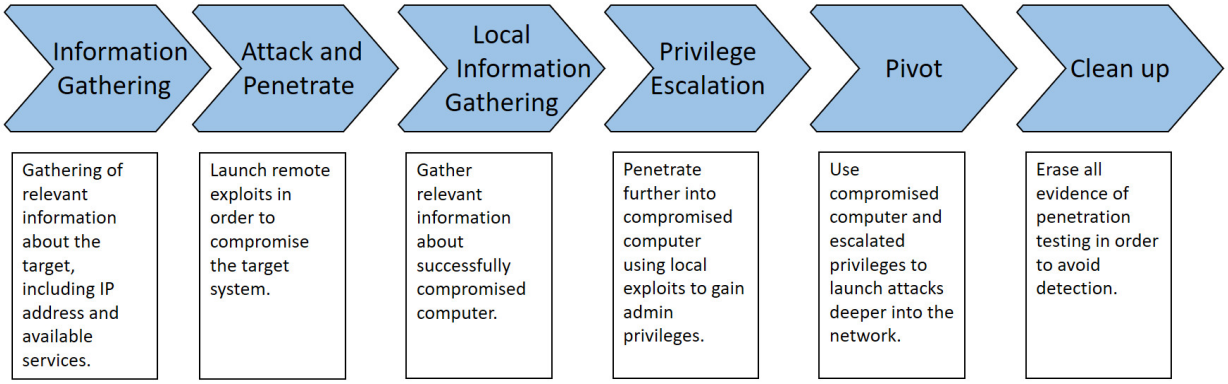

Figure 3.1: Penetration Testing Methodology

Various organizations released free penetration testing methodology documents. The process we cover in this project target many topics that have been covered in these methodologies. Each one of these documents offers practical knowledge into pentesting from many different perspectives. Five of the most useful methodologies on testing are:

- *Open Web Application Security Project (OWASP) Testing guide*[1]
- *Penetration Testing Execution Standard (PTES)*[2]

[1] www.owasp.org
[2] www.pentest-standard.org

- *NIST Special Publication 800-115: Technical Guide of Information Security Testing and Assessment*[3]
- *Open Source Security Testing Methodology Manual (OSSTMM)*[4]
- *Penetration Testing Framework*[5]

In this project, we used a PTES methodology. *The Penetration Testing Execution Standard* contains the seven principal parts of the pentesting, which cover the detailed procedures of all the required steps of the process. In the following sections, we summarized every single step of PTES document .

3.3.1 Pre-engagement interactions

The pre-engagement interaction is the *"get out of jail free"* card. This is the planning stage, and it presents the techniques and the tools to be applied during the pentest process. During this phase, the pentester and the client must agree upon several technical and legal matters. The scope is identified during this phase, including the cost and the duration of the whole procedure. The penetration test can cause interruptions in system work-flow, so the assessment times need to be decided from the beginning. Besides, the objectives of the test must be discussed to avoid all risks and perform good results.

3.3.2 Reconnaissance

Once the test is scoped and all necessary agreements are signed, the pentest starts with the reconnaissance step. In this phase, the pentester collects information about the target from different public sources. The penetration tester needs to be familiarized with the target's people and culture, understanding the particular business vocabulary used by employees in the target organization. We try to find detailed information about the technical infrastructure of the target system, seeking hints about its architecture products, configuration, and design.

The recon phase is very important in performing successful penetration tests. Moreover, the information gathered through this phase will be effective during all the different pentesting phases and will be helpful in the production of the final report.

[3] www.csrc.nist.gov
[4] www.isecom.org/osstmm
[5] www.vulnerabilityassessment.co.uk

3.3.3 Scanning

In this section, we will discuss the goal of the scanning stage. Then we will process through different kinds of scans, including port scanning, version scanning, and network sweeps.

The penetration tester needs to use a methodological strategy of scanning to minimize the risk of damaging the target systems. Usually, there is a possibility that our interactions can slow the services or crash the OS in target machines.

The goal of the scanning phase is to understand the target's environment functions and discover openings by interacting directly with it. Special objectives in this stage goal consist of determining the target's address, including hosts servers or clients, used equipment (switches, routers, and firewalls). Furthermore, the pentester needs to identify the network topology and create a diagram to display the interconnection between the different network equipment. Then, the pentester tries to find the OS types of the different machines to ensure the activities of the following stage based on founded vulnerabilities in those units.

Next, the pentester should list the listening TPC/UDP ports of the target systems because every open port provides a possible road to compromise the system. Moreover, to identify the open ports, we need to confirm which is the listening port of each service and the versions of the running applications. Then, we need is to select the potential vulnerabilities based on all behavior of the system during the networking interactions.

To achieve the goal of the successful scan, the pentester perform many types of scans based on specific workflow, indicated in the Figure 3.2:

Figure 3.2: Workflow of the Scanning Phase

- Network sweeping: This type of scan discovers the used addresses by sending probe packets to all targets in the network range. And, every received response during the network sweep is a possible system using this particular address.
- Network tracing: This is a strongly related task to network sweeping, in which the pentester attempts to detect the target's topology network by drawing a detailed map about it.

- Port scanning: By looking for listening TCP/UDP ports, these sorts of scans try to find the possible openings in target systems. Open ports signify that a service is listening. If this service is vulnerable, the pentester may have to find a way to compromise the target.
- OS fingerprinting: Every operating system has different behavior in the network. These behavioral actions can be determined by creating specific packet tests. Using a technique called active OS fingerprinting, the OS types of target systems can be remotely identified. However, using some sniffing tools with a detect OS functionality, the pentester can passively identify the OS type without sending any packets but simply by receiving them.
- Version scanning: The pentester needs to recognize every listening service on all ports. Although numerous popular services listen on well-known ports, an administrator can load these services on other ports. During the version scan, the tester can check every protocol spoken in the network and identify the possible version of all services to given ports.
- Vulnerability scanning: In these type of scans, the pentester evaluate the target machines who have any potential vulnerabilities, which may include unpatched services or misconfigurations.

3.3.4 Exploitation

In this section, we will discuss the exploitation of target systems. By the end of the scanning phase, the penetration tester must take a listing of the gathered information and the different potential vulnerabilities discovered.

As we mentioned in Section 3.1.2, an exploit is a technique or executed code used by a threat to trespass a founded vulnerability on a target network. For an ethical hacker, the main objective of exploiting a target system is to gain access and run some commands on it.

The exploitation process may render limited privileges on the target system to execute set commands as low privilege users or give high privileges as a superuser like *UID 0* or *root* on Linux OS or *Administrator* on Windows OS.

3.3.5 Post-exploitation and Pivoting

In the previous section, we discussed the exploitation phase so far is made basically to allow the pentester to gain shell access to a target system, such as /bin/sh, Meterpreter, or cmd.exe access. But following to access a target, what do you do with it?

3.3.5.1 Post-exploitation

The activities executed by the tester after exploiting the target system are known as post-exploitation activities. They express the uses of shell access effectively to explore the target machine, get its resources to attack other systems, and create pivot attacks included in the scope.

The objective of post-exploitation exercises is to build a better understanding of the risks presented by the discovered and exploited vulnerabilities. However, the pentester needs to stay in scope and follow the details assigned in the Rules of Engagement, and this is very important in the post-exploitation phase

3.3.5.2 Pivoting

Pivoting is the method used by pentester in a compromised system to move to other hosts on the network. Figure 3.3 describes an example of this type of attack, if the attacker compromises the machine of IP:192.168.78.5, moving to 192.168.78.25 computer becomes easier. These strategies are usually named *multi-layered* attacks. [10]

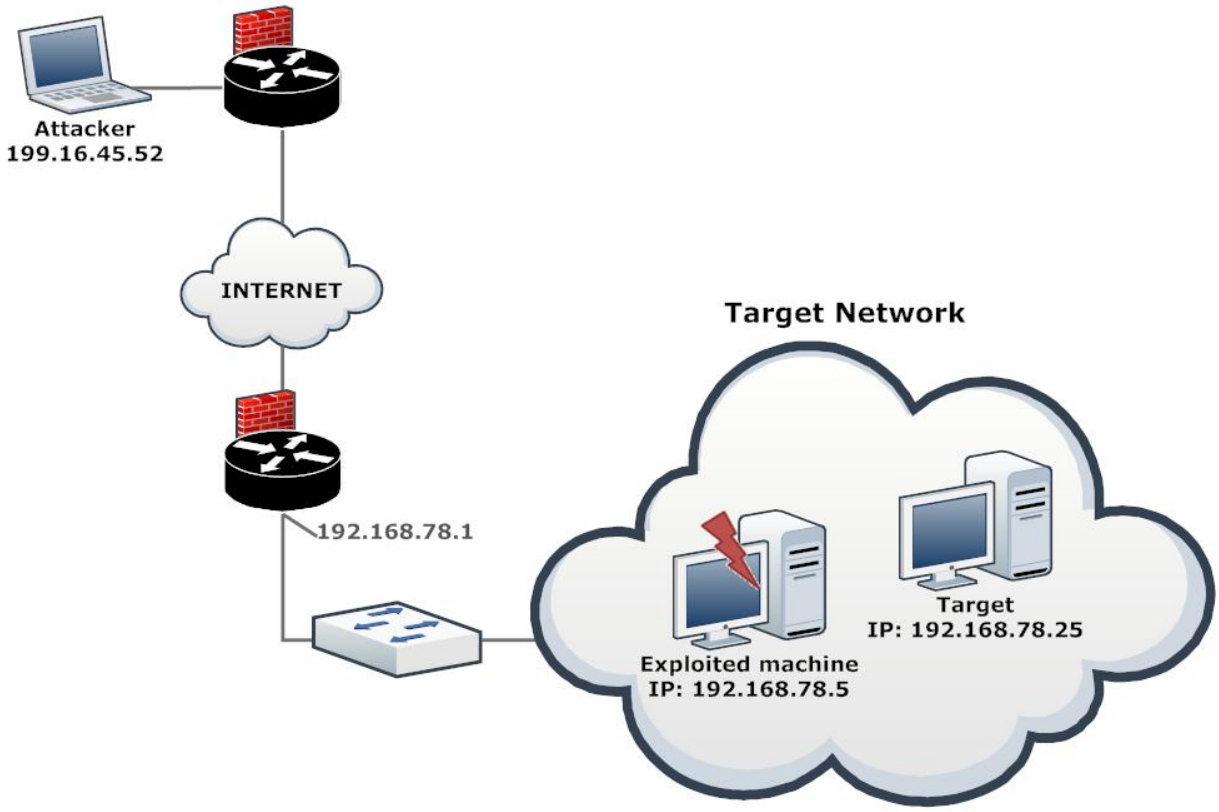

Figure 3.3: Scenario of Pivoting Chain

Chapter 4

Asynchronous Advantage Actor Critic (A3C)

In this section, we will explain the machine learning models that can be implemented to automate to penetration testing process. There are a variety of machine learning models available, but we need to adopt a model that is suitable to realize the goal.

4.1 Reinforcement learning background

There are several approaches to achieve automation with machine learning. For example, one approach is to use the result of prior human exploitation as teacher data, and let the machine learn how to do the method of exploitation by supervised learning. However, there are a wide variety of target hosts types, open port numbers, running products and version. In addition, if we add the combination patterns such as exploit modules and payloads provided by Metasploit, a huge amount of teacher data is required. For this reason, it is not practical to adopt supervised learning in this project.

4.1.1 CartPole

CartPole is a game of balancing a Pole (a brown stick) on top of a Cart (a black box). Figure 4.1 describes the moving of the Cart left and right according to the inclination of the Pole so as not to knock it over.

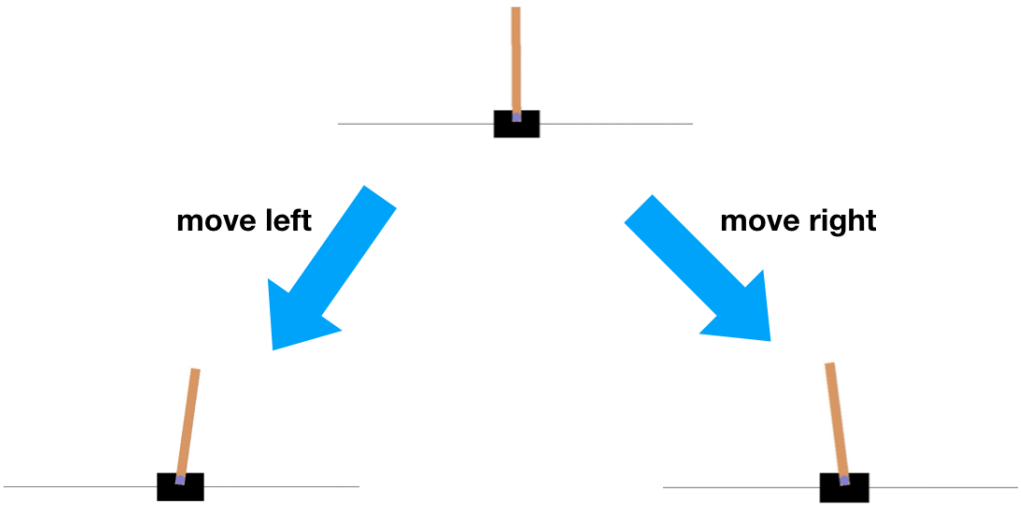

Figure 4.1: Reinforcement Learning with Cart-Pole

In this example, we use CartPole, which is included in OpenAI's *Gym*[1] package for evaluating reinforcement learning models.

A3C can know the current state through Gym. In CartPole, the *state* is defined as follows:

- Cart's position.
- Cart's velocity.
- Pole angle.
- Pole's angular velocity.

Then, depending on the current state, some *action* is selected. In CartPole, the action is defined as follows:

- Move to the right.
- Move to the left.

Finally, a *reward* is given according to the result of the action. CartPole defines rewards as follows:

- Positive reward (+1): when the Pole is balanced.
- Negative reward (-1): When a Pole is defeated.

[1] gym.openai.com

By designing *state*, *action*, and *reward*, the system gradually learns the actions that are rewarded according to the state. In addition, when Cart acts, the state such as *Cart's position* or *Pole's angle* changes, and this changed state is defined as the *next state* while the *next state* is set as the *current state* for the next action. By chaining the states in this way, it becomes possible to continuously evaluate the goodness or badness of the behavior from the beginning of the task to the present.

4.2 Asynchronous Advantage Actor Critic

In addition to supervised learning, there are other approaches to machine learning such as unsupervised learning and reinforcement learning. Reinforcement learning is a *self-learning* method through a large amount of trial and error of behavioral patterns to successfully perform a task and does not require any teacher data. If we take advantage of this property, we may be able to realize the automated pentesting process.

Therefore, as a model of reinforcement learning, we will use the Asynchronous Advantage Actor (A3C) model proposed in the *Asynchronous Methods for Deep Reinforcement Learning* published by DeepMind in 2016[11]. DeepMind[2] is a subsidiary of *Alphabet Inc*, which is famous for developing the Deep Q-Network (DQN)[5], which can self-lean the rules of games such as BreakOut and Invaders games, as well as AlphaGo.

Since reinforcement learning is based on the principle of reward systems, it requires many trials of various behaviors in various states. In other words, it takes a lot of time to learn.Therefore, A3C realizes a signigicantly faster learning time by distributed learning with multiple agents by asynchronous. According to DeepMind's verification, A3C trained on 16 CPU cores is much faster than DQN trained on Nvidia K40 GPU. [11]

The learning method of A3C is almost the same as DQN, but with the following three main improvements:

1. Advantage
2. Actor-Critic
3. Asynchronous

[2]www.deepmind.com

4.2.1 Advantage

In normal Q-learning and DQN, the Q-function was updated so that Q(s,a) approaches $r(t) + \gamma.maxQ(s', a)$. Here $r(t)$ is the reward obtained at time t, γ is the time discount factor, and s is the state resulting from the action of a in a state s', in other words ($s' = s(t+1)$). The Q-function recursively implemented by $Q(s, a) \rightarrow r(t) + \gamma.maxQ(s', a)$.

Advantage is the idea of updating the Q-function by moving it tow or more steps ahead instead of one step. For example, the advance considering two steps ahead is as the follow: $Q(s, a) \rightarrow r(t) + \gamma.r(t + 1) +^2 .maxQ(s', a)$. At time t, action $a(t)$ is performed and the state s changes and got a reward $r(t)$. Next, perform some action $a(t + 1)$, the state becomes s', and the reward $r(t + 1)$ is obtained. This time $s' = s(t + 2)$.

However, the using of *advantage* parameter all the way can not help during the whole automation stages. Because, the Q-function is used to determine the $a(t + 1)$ in the middle of completion's process. And, if this action is wrong, it will be the same during all the future cases too. Therefore, the using of *advantage* one after other can cause many error in the machine learning model, so it is better to balance to use it up a little ahead.

4.2.2 Actor-Critic

Q-learning and DQN outputs the total reward R (discounted total reward) in state s, which is the sum of the rewards that will be obtained in the future by time-discounting. Reinforcement learning was carried out using the Q-function of $R = Q(s, a)$, where Q-function is called the *Value-Based*.

On the other hand, *Actor-Critic* is a combination of value-based and another framework called Policy-Based. *Policy-Based* is a method to determine the action a directory from the state s without funding the Q-function. For example, AlphaGo Zero is a Actor-Critic that combines Policy-Based and Value-Based in one network.

The following Figure 4.2 describe the case of Actor-Critic. The network is divided into an Actor part that outputs the action and Critic part that outputs the discount total reward R of the state s.[6]

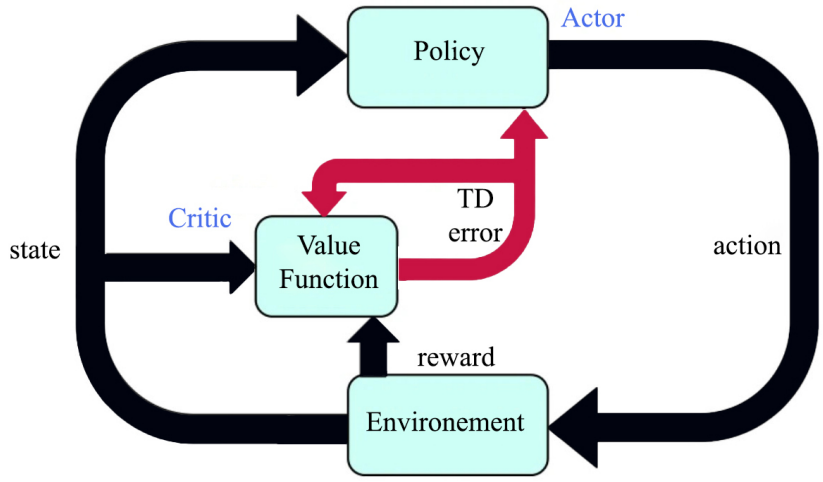

Figure 4.2: The Actor-Critic Architecture

In this implementation of *actor-critic* we used the process of CartPole's network[3], which is represented in Figure 4.3.

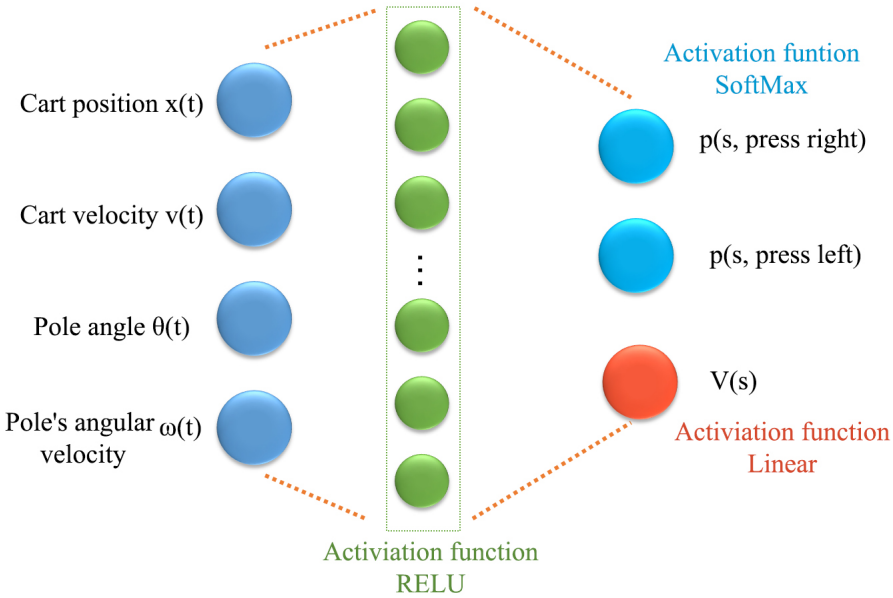

Figure 4.3: Actor-Critic Neural Network in CartPole

There are four input elements, each element of the state $s(t)$ is input. The state $s(t) = x(t), v(t), \theta(t)$, and $\omega(t)$, which represents respectively the position, speed, rod angle, and angular velocity of the cart.

In addition, there are three output elements: action a = [push the cart to the right, push card to the left] (Actor) , and output discount total reward $R = V(s)$ in state s (Critic). The two Actor elements show the probability $p(right)$ that is better to push to the right and the probability $p(left)$ is better to push the card to the left in state s where $p(right) + p(left) = 1$.

[3]gym.openai.com/envs/CartPole-v0/

Then, in the case of the state s at time t, the action a is determined to be right or left according to this probability p. Also, it is common for Actors and Critics to share the process up to the final output layer.

To explains the flow of determining action a from input s on the Actor-Critic network the learning method of the backward flaw need to be defined, which is the weighting factor that connects the elements of the network. For this Critic, the $Qfunction$ has updated to the value function $V(s)$ to $V(s) \rightarrow r(t) + \gamma.V(s')$.

Considering Advantage, $V(s) \rightarrow r(t)+\gamma.r(t)+(y^2).V(s')$, then the actor side also needs to be updated. It is known that the update on the Actor side should be update so that $log(\pi_\theta(a|s).A(t))$ increased. This is known as the *Policy Gradient Theorem*, where $A(t) = R((t) - V(s))$, and $A(t)$ must be treated as a constant when updating the parameters. For example, if the function $a(t)$ = push to the right is performed at time t and the state is s then the result is $log(p(right).(R(t) - V(s)))$.

Since the expression to be maximized contains the term of $p()$, the network weight on the Actor side can be updated.

However, since TensorFlow cannot be maximized, the network weight is updated in the direction of minimizing the minus. Also, at the time of implementation, an entropy term is added to prevent $p()$ from being updated at once and falling into the local minimum, making it difficult to converge.

4.2.3 Asynchronous

Asynchronous is multi-agent distributed learning. If prepares multiple learning environments with multiple threads and the Agents in each environment accumulate experience on their own. Each thread has it own Advantage-Actor-Critic network. In addition, there is a Advantage-Actor-Critic network (Parameter Server) shared by all threads. Furthermore, each agent seeks the direction (gradient) to update the weight of the network with Advantage-Actor-Critic so that more rewards can be obtained from the experience accumulated by moving freely.

In this implementation, the agent does not update its network weight, but passes the direction (gradient) that causes the *Parameter Server* shared by all threads to update the network weights and continues the simulation.

Each thread performs these asynchronously at an arbitrary timing. In other words the flow is as follows:

1. The thread copies network weights from Parameter Server
2. The thread's agent inputs s into its network to get a
3. Execute a, and get $r(t)$ and s'
4. Store (s, a, r, s') in thread memory
5. Repeat the steps ❷ and ❹ until the $Tmax$ step elapses or the end is reached in each thread
6. When enough experience has been accumulated, use the contents of the thread's memory to obtain grad, the direction in which the network weights are to be updated.
7. Pass grad to Parameter Server
8. The Parameter Server updates the network of the Parameter Server in the grad direction.
9. Return to first step (❶).

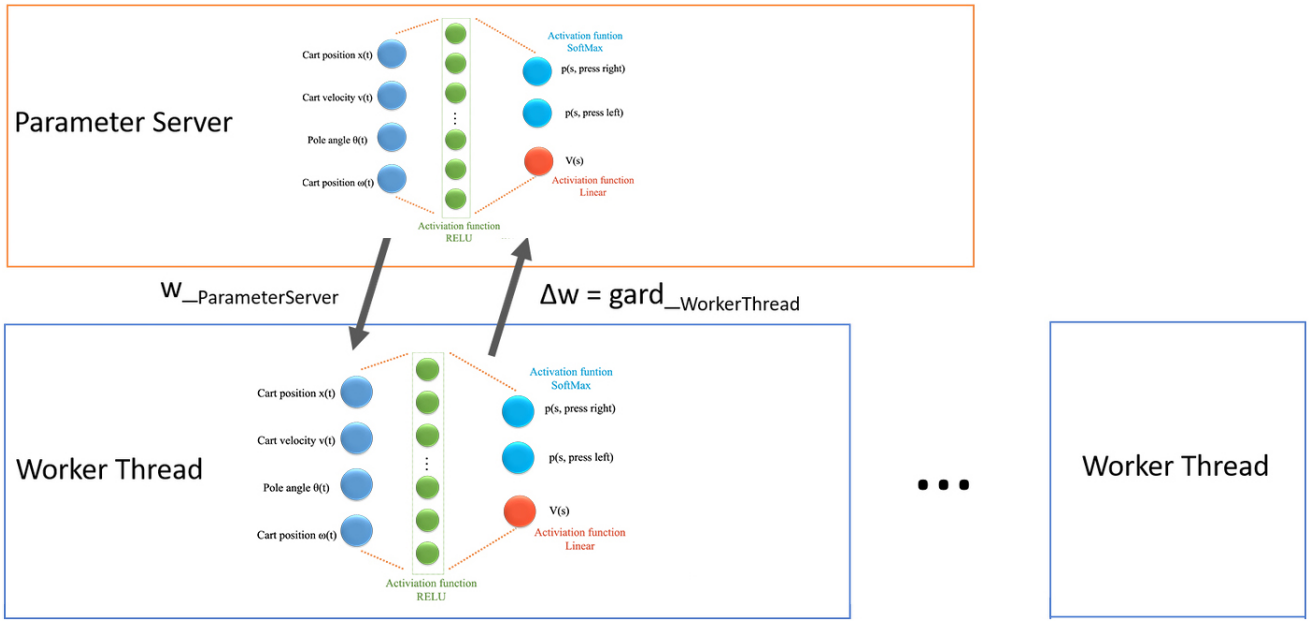

Figure 4.4: Asynchronous Learning

Chapter 5

Experimental Results

5.1 The network design

The network design of the attack simulation covers all the physical and logical links to model the attacks and packets flows on the network.

5.1.1 Typology

The network topology describes the connection between the various components and hosts in the network. In the testing typology, we have some subnets that can change data directly in the internal network while others can not connect with an external network. For example, in figure 5.1, subnet 1 is the exclusive network that can change data with the external world, while the other subnets (2, 3, and 4) can not establish any connection with external networks. In this way, if the tester compromises any machine in subnet 1, he can pivot through other hosts to achieve the target machines (example: Active Directory).

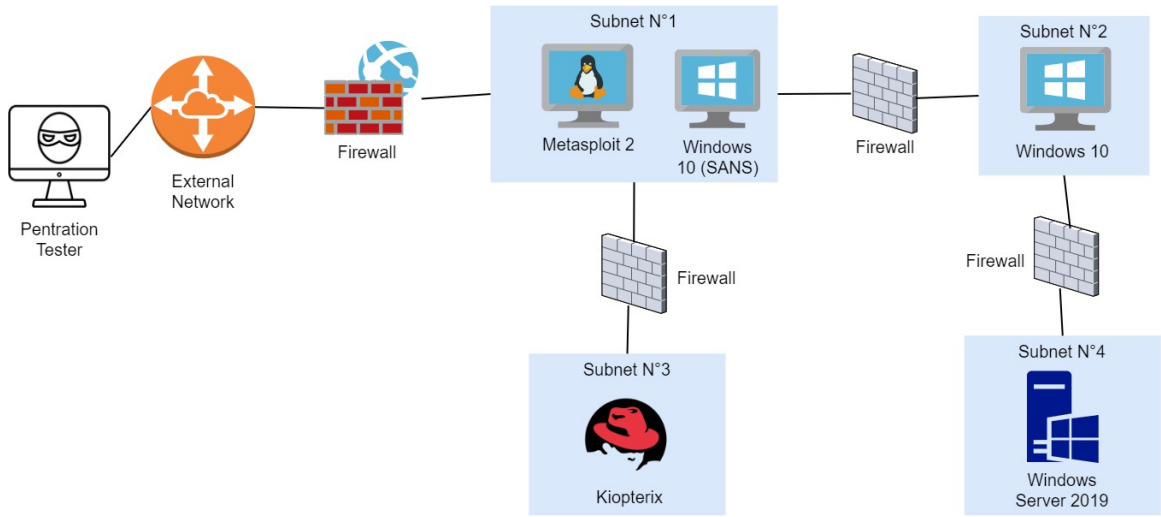

Figure 5.1: Example network with four subnets

5.1.2 Machines and Implementation

Usually, RL algorithms need a very large number of episodes for training and testing. For this purpose, we need a fast simulation of the environment. Additionally, the penetration testing process and the exploit methods need to execute in performing computers.

Thus, we experimented the testing process using two methods: exploit target system using Metasploit without machine learning interaction and simulate the attack using the Metasploit with A3C algorithm, defined in chapter 4. For each of them, we deployed in cloud service (AWS and NVIDIA)[1] a different OS as follow:

1. Three vulnerable OS.
    - Metasploitable2[2]
    - Metasploitable3
    - Windows 10 x64bit professional (modified by SANS)[3]
2. Two normal OS (Real-world scenarios).
    - Windows 10 professional x64bit
    - Windows Server 2019 x64bit

5.2 Metasploit

Metasploit framework is one of the auditing tools used in penetration testing and vulnerability assessment. It is a command-line tool that includes various auxiliary tools for port scanning, exploit execution and vulnerability assessment.

```
# cowsay++
 ------------
< metasploit >
 ------------
       \   ,__,
        \  (oo)____
           (__)    )\
              ||--|| *
msf >
```

Listing 5.1: Metasploit framework

[1] aws.amazon.com/nvidia
[2] sourceforge.net/projects/metasploitable
[3] sans.org

In this project, we used a ***Metasploit 4.16.48-dev***[4], which is installed by default on ***Kali Linux 2018.2***[5].

5.2.1 Cooperation with external programs

The general usage of Metasploit for humans is to enter the command from *msf-console*.But, since the purpose of this project is to *automate the penetration testing using machine learning*, we need to replace **human command input** with **machine learning command input**.

The Metasploit framework is developed basically in Ruby, which is the simplest method to use the API. However, other languages that support MessagePack and HTTPS, like Python, Java, and C, can be used to get the features of the RPC API.[6]

Therefore,A **RPC** service is a package of message types and remote processes that produce a organized method for external programs to communicate with web applications.In Metasploit The **RPC API** is implemented to execute its commands and get their results from the machine learning model using HTTP-based remote procedure call (RPC).

5.2.1.1 Enabling the RPC API

The RPC API is disabled by default, but it can be enabled by loading the *msgrpc* plugin. The following shows an example of enabling the RPC API (starting the RPC Server) with the IP address as *192.168.1.10*, Port as *55552*, User for authentication as *user*, and Password as *pass123*. [7]

```
[*] MSGRPC Service: 192.168.1.10:55552
[*] MSGRPC Username: msf
[*] MSGRPC Password: abc123
[*] Successfully loaded plugin: msgrpc
```

Listing 5.2: Enabling the Metasploit RPC API

[4]www.metasploit.com
[5]www.kali.org
[6]docs.rapid7.com/metasploit/rpc-api/
[7]ocs.rapid7.com/metasploit/rpc-api/

By default, Metasploit sets default options if the user does not specify them. To customize these settings, with the *load* command the user can define the hostname, username, password, and port to connect to the server:

- *ServerHost* - The hostname listened by the server.
- *ServerPort* - The port listened by the server.
- *User* - The username to login the server.
- *Pass* - The password to login the server.
- *SSL* - By default, the SSL id disabled, the user can enable it using *true*

5.2.1.2 Manipulating Metasploit with RPC API

By enabling the RPC API, various commands can be sent to Metasploit from the outside. Many APIs are implemented in Metasploit, such as *auth.login* to log in to Metasploit, *console.create* to start *msfconsole*, *console.write* to send command to *msfconsole*, and *console.read* to receive the execution results from the command's output.

The following is an example of using the RPC API on the *Python console (3.6.5)*[8]. In this example, after logging to Metasploit, the *version* command is sent to get the results of its execution.

In [1]:
```
# Import MessagePack and package for HTTP communication
import msgpack
import http.client

# Define Metasploit connection information
host = '192.168.1.17'
port = 55553
uri = '/api/'
headers = {'Content-type': 'binary/message-pack'}

#Connect to Metasploit
client = http.client.HTTPConnection(host, port)

# Login to Metasploit using the auth.login API
user = 'user'
password = 'pass123'
option = [user, password]
option.insert(0, 'auth.login')
option
```

Out[1]:
```
['auth.login', 'user', 'pass123']
```

In [2]:
```
params = msgpack.packb(option)
client.request('POST', uri, params, headers)
ret = client.getresponse()
response = msgpack.unpackb(ret.read())
response
```

Out[2]:
```
#If login is successful, authentication token will be returned
{b'result': b'success', b'token': b'
    TEMP3gTY7V2egVFx6vbaerT2O5SvFe9T'}
```

Listing 5.3: Authenticate to Metasploit using RPC API

[8] www.python.org

```
In [3]:
# Create MFSconsole with console.create API
token = response.get(b'token')
option = [token]
option.insert(0, 'console.create')
option

Out[3]:
['console.create', b'TEMP3gTY7V2egVFx6vbaerT2O5SvFe9T']

In [4]:
params = msgpack.packb(option)
client.request('POST', uri, params, headers)
ret = client.getresponse()
response = msgpack.unpackb(ret.read())
reponse

Out[4]:
# If the MSFconsole is successfully created, the console id
    returns
{b'wrote': 7}
```

Listing 5.4: Creation of MFSconsole using console.create API

With the same method, we can create and execute the *console.read* and *console.write* APIs. Thus, by using the RPC API, Metasploit can be easily manipulated from Python.

5.3 APTML Application

In this section, we will explain the logic of The *APTML* application, which automates a series of actions from exploration to intrusion against a target host by linking Metasploit and A3C implemented using Python.

The following figure 5.2 describe the system component of APTML application.

This time, we created a unique exploitation tool called **APTML** referred to *Automated Penetration Testing Using Machine Learning*, it starts by search to exploit the target host by linking Metasploit and machine learning.

APTML in current version, can do the following actions:

- *Intelligence gathering*: Collect information on target hosts
- *Modeling threat*s: Identify known vulnerabilities
- *Vulnerability analysis*: Determine effective assessment method
- *exploitation*: Execute exploit with the determined assessment method
- *Post-exploitation*: If the exploitation is successful, attempt to invade another hosts
- *Reporting*: Test result reporting

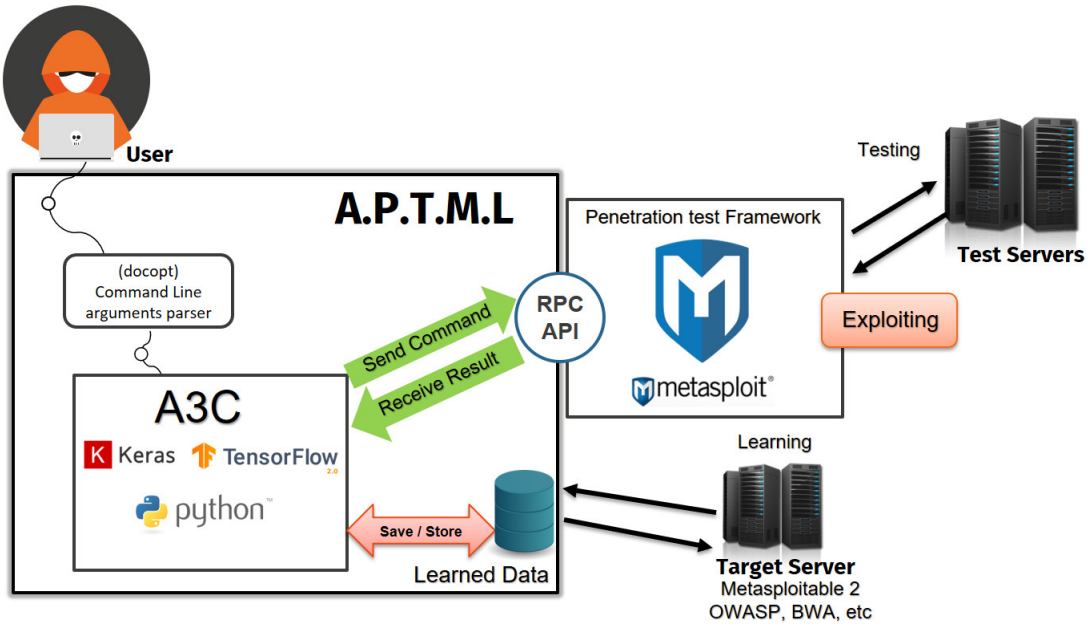

Figure 5.2: The system component of APTML

APTML operates in two modes, **learning mode** and **production mode**. In the learning mode, various *exploit modules* implemented in Metasploit are run by trial and error, referring to the status of the target host (OS type, open ports, product name running). Then, it learns the modules that succeed and fail in exploitation for various *states*. When learning is complete, weights are assigned for each exploit module for each *state*. Note that no supervisory data is used in the training, more details are described in Chapter 4.

In the production mode, based on the learned result, the exploit module with the highest probability of successful exploitation (high weight) for the *state* of the target host is executed with pinpoint accuracy. Therefore, it is possible to achieve a successful explosion with a minimum of one trial.

The learned result can be stored. Then, by using a variety of hosts with different OS types, the open port numbers, running products, flexible exploitation can be performed in response to various conditions. The more you use it, the more accurate the exploitation becomes. However, the following figure 5.3 described the conceptual diagram of APTML process of learning and training with the same basic as CartPole, but there are five difference.

1. Agent tasks
2. Input information to the neural network (=currnt state=
3. Output from the neural network (=action)
4. Rules for giving rewards

5. Models of the neural network

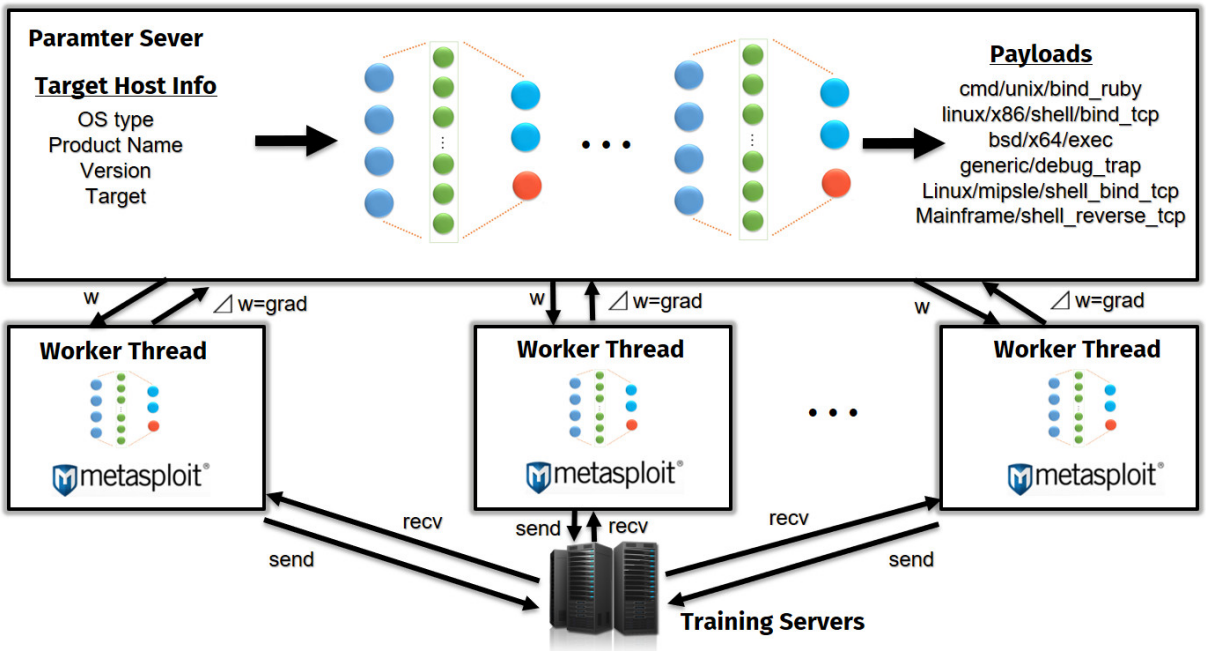

Figure 5.3: Conceptual diagram of APTML

5.3.1 Agent taks

In CartPole, the goal was to keep the balance while moving the cart left and right to avoid toppling the Pole. On the other hand, APTML sends the exploit module to the target host via Metasploit intending to make a successful exploitation. Thus, since the tasks are very different from CartPole, we need to design a *current state*, *action*, and *reward* considering the difference in tasks.

5.3.2 Current state

APTML defines the following seven types of information as *current state*.

1. OS type of the target host.
2. Port number to be inspected.
3. Protocol type (TCP/UDP).
4. Product name running on the port.
5. Product version.
6. Types of exploit modules available in Metasploit.
7. The target that can be selected for the exploit module.

Using A3C, APTML can learn the optimal behavior for the current state. Here, the optimal behavior is the behavior that **exploitation succeeds**. To increase the probability of successful exploitation, it is necessary to consider the seven defined types of information of the target host and use an appropriate payload.

The following sections explain how to acquire each type of defined information.

5.3.2.1 Acquisition of OS type

Using the RPC API *console.write*, a port scanning is performed on the target host using the *db_nmap* command. Then, after the port scanning, the *hosts* command is executed. The following shows the output of *hosts* command after port scanning the target host = 192.168.1.19.

```
Hosts
=====
address       mac                name  os_name  os_flavor  os_sp  purpose
-------       ---                ----  -------  ---------  -----  -------
192.168.1.19  00:0C:29:0D:DC:28         Linux              2.6.x   device
```

Listing 5.5: Acquisition of OS type

5.3.2.2 Acquisition of "port number", "protocol", "product name", and "version"

Using the same RPC API *console.write*, the *services* command executed to get the following result.

```
Hosts
=====
host          port  proto  name          state  info
----          ----  -----  ----          -----  ----
192.168.1.19  21    tcp    ftp           open
192.168.1.19  22    tcp    ssh           open
192.168.1.19  23    tcp    telnet        open

...snip...

192.168.1.19  2049  tcp    nfs           open
192.168.1.19  3306  tcp    mysql         open
192.168.1.19  5432  tcp    postgresql    open
```

Listing 5.6: Acquisition of "port number", "protocol", "product name", and "version"

5.3.2.3 Obtaining "exploit Module Types"

The *module.exploits* can be obtained with RPC API in Metasploit

```
{b'modules': [b'windows/vnc/ultravnc_viewer_bof',
  b'windows/vnc/winvnc_http_get',
  b'windows/wins/ms04_045_wins',
  b'windows/vpn/safenet_ike_11',

... snip ...

 b'android/meterpreter/reverse_http,
 b'android/shell/reverse_http'
 b'android/shell/reverse_https',
 ']}
```

Listing 5.7: Exploit Module Types

Since the exploit module associated with the product name is determined in Metasploit, one exploit module to be set in the "current state" is randomly selected from those associated with the product.

5.3.2.4 Get "exploit target"

Using the the RPC API *console.write*, we can select with *use* command the exploit module and set the *current state*, then *show* the targets, and execute the command to get it.

The following shows the result of executing *use exploit/linux/http/linksys_apply_cgiafter* command then *show targets*.

```
exploit targets:
   Id  Name
   --  ----
   0   Generic
   1   V. 1.42.2
   2   V. 2.02.7_ETSI
   3   V. 3.03.6
   4   V. 4.00.7
   5   V. 4.20.06
```

Listing 5.8: exploit target

Executing the *show targets* command will output the list of targets available for the selected exploit module.

Since the number identifying the target is output in the column "Id", use the RPC API "console.read" to get the command execution result from the buffer, and parse the corresponding part (Target ID) as "exploit target". Note that there may be multiple targets, in which case one target is randomly selected and set to the current state.

As described above, APTML mainly uses the various information of the target host obtained by port scanning as the "current state". The auxiliary module provides more detailed information on the target host status, which will be implemented in the next version.

5.3.3 Action

In APTML, the payloads that can be referenced from the selected exploit module in the current state are defined as *actions*.

Besides, Metasploit used in this verification has 536 types of *payload*, but some exploit modules do not use any payload, so the number of actions sets to 536 + 1 (504 types).

The list of payloads that can be referenced from the exploit module is obtained using the RPC API *module.compatible_payloads*. The following shows an example of obtaining the list of payload names that can be referenced from the *"exploit/linux/http/linksys_apply_cgi"*module.

```
Compatible payloads
===================
Name                                  Rank    Description
----                                  ----    -----------
generic/shell_reverse_tcp             normal  Generic Command Shell
generic/shell_bind_tcp                normal  Generic Command Shell
generic/custom                        normal  Custom payload
linux/mipsle/shell_reverse_tcp        normal  Linux Command Shell
linux/mipsle/meterpreter/reverse_tcp  normal  Linux Meterpreter
linux/mipsle/shell/reverse_tcp        normal  Linux Command Shell
linux/mipsle/exec                     normal  Linux Execute Command
linux/mipsle/shell_bind_tcp           normal  Linux Command Shell
linux/mipsle/reboot                   normal  Linux Reboot
```

Listing 5.9: payload Lists

5.3.4 Remuneration

APTML defines *reward* as follows:

- Positive reward (+1): If the exploitation is successful.
- Negative reward (-1): If the exploitation is fails.
- No reward (0): When it is note related to the success or failure of exploitation, such as when an error occurs inside Metasploit.

5.3.4.1 exploitation success or failure judgment

The success or failure of an exploitation can be easily determined by combining the RPC API *module.execute* and *session.list*.

APTML uses *module.execute* to execute exploit, and this API returns *job_id* and *uuid* as return values, where *uuid* is a unique value associated with the executed exploit module.

Then, the execution of *session.list* returns the list of currently active session information (the list of sessions that have been opened after successful exploitation). Each session information contains an element *exploit_uuid*, which is the *uuid* of the exploit module that successfully completed the exploitation.In other words, by comparing the *uuid* obtained by *module.execute* with the *exploit_uuid*, we can easily determine whether the exploitation was successful or not.

As described above, by manipulating the functions provided in Metasploit via RPC API, we can obtain *state*, *action*, and *reward* necessary for A3C learning.

5.3.5 Neural network model

The neural network used in CartPole was a three-layer multilayer perceptron with the following number of nodes.

- Input layer: 4 nodes.
- Intermediate layer: 16 nodes.
- Output layer: 2 nodes.

As mentioned above, the number of input nodes = the number of states, and the number of output nodes = the number of actions.

On the other hand, APTML takes 7 states as input and 536 actions (payload) as output. Therefore, the neural network has the following structure.

- Input layer: 7 nodes.

- Intermediate layer 1: 50 nodes.
- Intermediate layer 2: 100 nodes.
- Intermedaite layer 3: 200 nodes.
- Output layer: 504 nodes

The activation function of the intermediate layer is ReLU (Rectified Linear Unit, Rectifier), and the activation function of the output layer is Softmax.

Moreover, the configuration of the neural network is a combination of the author's trial and error to achieve the highest accuracy of exploitation. It is subject to change depending on future verification.

5.4 Testing APTML and Results

In this section, we will explain the learning and testing process of APTML in action.

5.4.1 Learning mode

First, figure 5.4 describes the port scanning process. Port scanning is performed on the learning host and the *current state* is defined based on the resulting information of the target host.

```
[+] Execute Nmap against 192.168.1.16
[*] nmap -p0-65535 -T5 -Pn -sV -sT --min-rate 1000 -oX nmap_result_192.168.1.16.xml 192.168.1.16

[*] Start time: 2021/04/10 22:34:49
[*] Port scanning: 192.168.1.16 [Elapsed time: 0 s]
[*] Executing keep_alive..
[*] End time  : 2021/04/10 22:34:53
[+] Get port list from nmap_result_192.168.1.16.xml.
[*] Getting 21/tcp info: vsftpd 2.3.4
[*] Getting 22/tcp info: OpenSSH 4.7p1 Debian 8ubuntu1 protocol 2.0
[*] Getting 23/tcp info: Linux telnetd
[*] Getting 25/tcp info: Postfix smtpd
[*] Getting [illegible] info: ISC BIND 9.4.2
[illegible]
[*] 66/68 Accessing : Status: [illegible]2.16:8180/webmin/
[*] 67/68 Accessing : Status: 404, Url: http://192.168.1.16:8180/phpbb/
[*] 68/68 Accessing : Status: 404, Url: http://192.168.1.16:8180/dwr/index.html
[*] Analyzing port 21/tcp, vsftpd/2.3.4, Available exploit modules:1
[*] Analyzing port 22/tcp, ssh/4.7p1, Available exploit modules:15
[*] Analyzing port 23/tcp, telnet/0.0, Available exploit modules:9
[*] Analyzing port 25/tcp, postfix/0.0, Available exploit modules:1
[*] Analyzing port 53/tcp, bind/9.4.2, Available exploit modules:1
[*] Analyzing port 80/tcp, apache/2.2.8, Available exploit modules:20
[*] Analyzing port 111/tcp, rpc/0.0, Available exploit modules:18
[*] Analyzing port 139/tcp, samba/4.X, Available exploit modules:8
```

Figure 5.4: Learning mode: Port Scanning

In the learning mode, the exploit, which is described in figure 5.5, is executed using various payloads while randomly recombining the states (port number, product name, exploit module and target, etc.). Then, It accumulates rewards for each state and action.

Figure 5.5: Learning mode: Exploit process

5.4.2 Production mode

In the production mode, after Port scanning is executed on the target host to obtain the state, exploit is executed based on the learned results with the combination of "Exploit module/Payload/Target" that has the highest probability of successful Exploitation for the obtained state. If the Exploitation fails, it will not be executed. If the exploitation fails, the next combination with the highest probability will be tried (maximum number of attempts is 5).

Figure 5.6: Learning mode: Production phase

If the exploitation is successful, the message "CHECKMATE" will be displayed. At the same time, a message that the session is open is displayed on the MSFconsole on Kali Linux.

As you can see from the figure 5.6, the exploitation is successful in pinpointing several port numbers and products. Let's take a look at the specific exploit contents for some of the ports.

- 23/tcp, telnet - For all 455 patterns of Exploit module/Payload/Target combinations, we selected the combination "Exploit: /solaris/telnet/fuser, Payload: cmd/unix/bind_perl, Target: 0", and on the first attempt Exploitation succeeds on the first trial.
- 445/tcp, samba - For all 991 patterns of Exploit module/Payload/Target combinations, we selected the combination "Exploit: /multi/samba/usermap_script,

Payload: cmd/unix/bind_perl, Target: 0", and Exploitation succeeded on the first Exploitation was successful on the first attempt.

- 80/tcp, apache - For all 5922 combinations of Exploit module/Payload/Target, we selected the combination "Exploit: multi/http/struts2_rest_xstream, Payload: cmd/unix/bind_perl, Target: 0". and succeeded in Exploitation on the fourth attempt.

By learning, we were able to probabilistically select the best combination of the 991 and 5922 possible Exploit module/Payload/Target combinations and execute Exploit with pinpoint accuracy. This result shows that learning can be used to achieve an efficient Exploit without wasting time.

We also found that we were not able to successfully Exploit targets such as "22/tcp, ssh" and "1099/tcp, java-rmi" that had not been successfully Exploited or had a small number of successes (not imprinted in memory) during training. We believe that these problems can be improved with further training.

5.4.3 APTML Results

In this Section we provide the result of some experiments run with and without APTML application to test the security of different hosts in different networks and DMZ.

To test the performance of APTML, we designed and implemented several systems in the different networks defined in section 5.1. In the first stage, we tried to evaluate the attack using only the Metasploit framework without any machine learning interactions. Then, we implemented the APTML application based on machine learning to test the same system in the same environment. The following two tables describe some result values from the APTML process and Metasploit only attacks during the experimental process. These values are mentioned in the following figure 5.7.

Step	0	50	100	150	200	250	300	350
Reward	0	7	12	41	46	72	98	132

Table 5.1: Rewards values of APTML during the experimental process

Step	0	50	100	150	200	250	300	350
Reward	0	2	3	14	22	24	32	39

Table 5.2: Rewards values of Metasploit only during the experimental process

As a comparison, we measured the number of rewards by steps based on the number of attacks in each cycle. We used one machine attacker for both evaluation tests. Figure 5.7 shows the evolution of total reward received (exploits results) by the A3C methods and the Metasploit only attack method. In A3C, we periodically evaluate the agent's performance and calculate the rewards. Alternatively, in the Metasploit without machine learning, we try to find the successful exploit in each step. The blue line shows the A3C exploit rewards, while the red line describes the Metasploit without machine learning evolution. In addition, X-axis represents the steps and iterations values while Y-axis represents the gain rewards for each step.

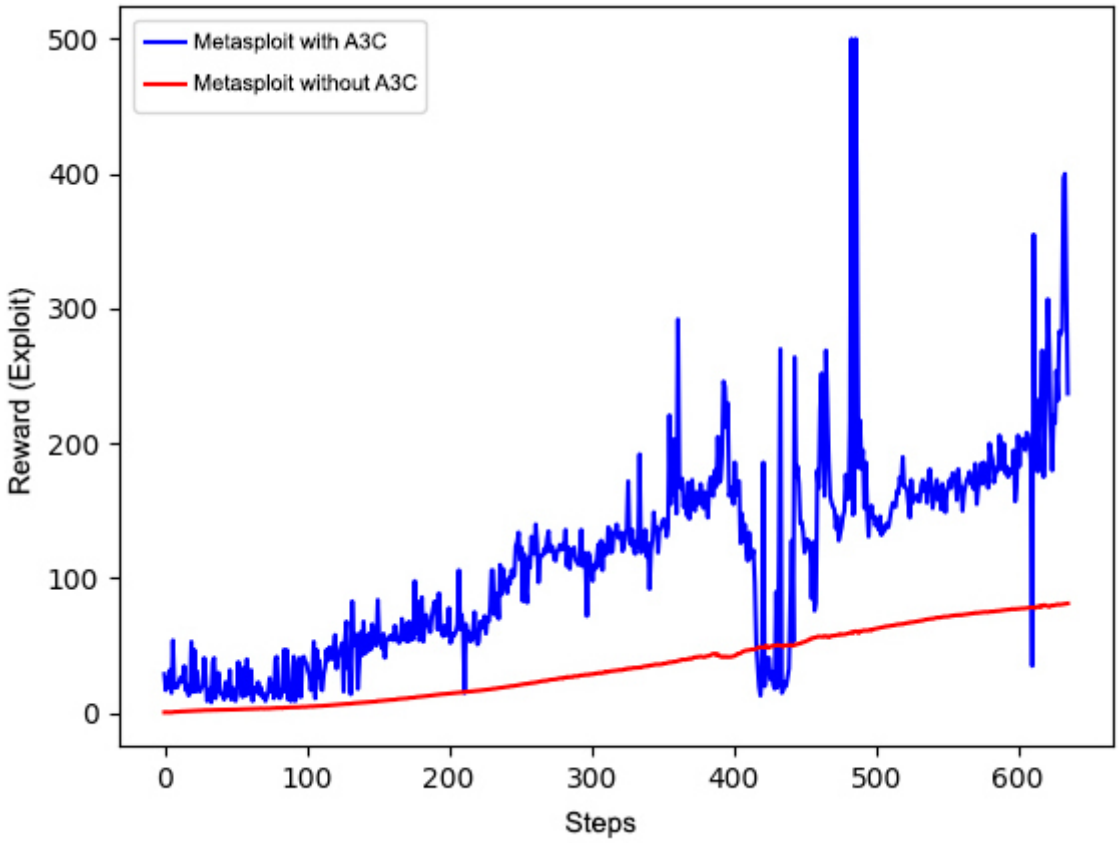

Figure 5.7: Metasploit Vs. Metasploit with A3C

Finally, we notice that the APTML and Metasploit only tests start with a small number of rewards in the first stages. Then we mention that the APTML application gradually rises faster than the Metasploit method after 150 iterations. After about 250 steps, the number of rewards taken by APTML becomes so heavy compared to Metasploit results.

5.4.4 Discussion

Although the A3C algorithm can converge quickly when using a variety of internal or DMZ typologies, we believe that further research is required to determine the best parameters for the A3C model when simulating broad attack scenarios with a variety of properties.

As a future step, we consider adding the A3C algorithms multiple training models with different results of previous attacks data. Thus, this model can analyze special features and attributes in different typologies with high performance and optimal attack paths.

Our system is intended for use in the field of cybersecurity education and training, and it can already be used for attack training activities, such as suggesting attack paths for trainees to try out on their own in the cyber range using a directed learning methodology.

Since the hacking techniques are growing day by day, we can not ensure that yesterday's security zones are safe today. Therefore, we need to rapidly find vulnerabilities and weaknesses then take action before evil hackers do.

Our APTML framework is planned for the cybersecurity education use and real systems security evaluations under the conduct of use and code of ethics[9]. We aim to also use APML to implement a realistic attack and help the white hackers in the purple teams to evaluate the defenses and intrusion systems capabilities.

5.5 Conclusion

While RL will become a powerful tool for many domains as technology advances, there are still a number of obstacles to overcome in automated penetration testing. Scaling the algorithms with the number of devices in the network and the number of exploits available to the agents, in particular. Additional research into designing more effective algorithms or decomposing the pentesting problem would greatly aid in the resolution of these issues.

This project was a basic implementation of the RL method into penetration testing. We confirmed that is possible to implement the reinforcement learning in the cybersecurity domain to generate a successful and optimal policy to exploit target machines in internal networks or DMZ. Taking this step further, we can use this policy to identify the weakness in the network components and improve their security. The most advantageous side of using RL over other machine learning methods is that no requirement

[9] www.eccouncil.org/code-of-ethics

for prior data or knowledge of the network topologies and system characteristics.

Besides the implementation of RL in cybersecurity, it will be a powerful tool in many other domains. In automate penetration testing, there are several obstacles to managing in future works. In particular, the development of more powerful algorithms or transforming the current one into efficient and optimized methods.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

As with many fields, automation is an evident choice for the rapid growth of any technology or tool. In this study, we wanted to search for a solution to challenging cybersecurity problems. Thus, we focused on penetration testing as an offensive side of this domain. We planned to implement the RL as AI approaches to automate the pentesting process. This calls for two stages, the first was the design and build the A3C algorithm to host the pentesting requirement. The second stage was to link this method with the Metasploit framework.

The main finding of this work was the number of exploits and rewards generated by the A3C algorithm within a simulated environment with different subnets and in DMZ or internal networks. We were able to successfully apply different types of attacks and exploit many systems without any prior knowledge of their characteristics and their position in the network. This offers a point of advantages over currents automated pentesting approaches. However, we also figured that the A3C needs powerful hardware to execute efficiently. Additionally, A3C relies intensely on providing high-fidelity data for training agents before they are capable of being applied in real-world scenarios.

6.2 Future Work

This project was intended to be a preliminary investigation into using RL for automated penetration testing, but it has resulted in a slew of new problems to address before this technology can be used in a commercial setting. To begin, we must improve the A3C algorithm in order to accelerate pivoting and enforce privilege escalation attacks. Then, use methodologies to locate a web application's vulnerability in the network systems.

Following that, the APTML framework can be used in a more practical environment with more network functionality and real-world settings.

This project was designed to be an initial study into using A3C for automated penetration testing and solving several cybersecurity issues. Therefore, this leads us to solve many problems before using it in a commercial environment. Firstly, we need to develop the A3C algorithm to move faster in pivoting and privilege escalation attacks. Then, we need to perform a methodology to find the web application's vulnerabilities hosted in the network systems. The next step after that will be to apply the APTML framework in a more realistic environment with more network features and real-world settings.

Bibliography

[1] M. Boddy, J. Gohde T.Haigh, and S.Harp. *"Course of action generation for cyber security using classical planning", in Proceedings of the Fifteenth International Conference on International Conference on Automated Planning and Scheduling, ICAPS'05, p. 12–21*. 2005.

[2] C. Sarraute J. L. Obes and G. G. Richarte. *"Attack planning in the real world" arXiv: Cryptography and Security"*. 2013.

[3] Andy Applebaum et al. *"Analysis Of Automated Adversary Emulation Techniques"*. 2017.

[4] Olivier Buffet Carlos Sarraute and J¨org Hoffmann. *"POMDPs Make Better Hackers: Accounting for Uncertainty in Penetration Testing"*. 2012.

[5] Volodymyr Mnih et al. *"Human-level control through deep reinforcement learning. Nature 518, 529–533"*. 2015.

[6] Riachard S. Sutton and Andrew G. Barto. *"Reinforcement learning: an introduction, p395-397"*. 2018.

[7] Monica Babes-Vroman M. Wunder M. Littman. *"Classes of Multiagent Q-learning Dynamics with ϵ-greedy Exploration"*. 2010.

[8] Andraw W.Moore L.P. Kaelbling Michael L. Littman. *"Reinforcement Learning: A Survey", Journal of Artificial Intelligence Research, vol. 4, May 1996, pp. 237–85*. 1996.

[9] Phillip L. Wylie nad Kim Crawley. *"The Pentester BluePrint: Starting a Career as an Ethical Hacker"*. 2020.

[10] Georgia Weidman. *"Penetration Testing: A Hands-on Introduction to Hacking"*. 2014.

[11] Volodymyr Mnih et al. *"Asynchronous Methods for Deep Reinforcement Learning"*. 2016.

Appendix

A.1 Preparing Penetration Testing Environment

In this section, we will walk through setting up the penetration testing lab to use multiple OS (Guests) inside the main OS (Host). This lab will be able to simulate an entire network architecture in a single physical computer.

A.1.1 Installing VMware

To start this lab, we need virtualization software.VMware Workstation Pro[1] is one of the most used environments for virtualization provided by VMware Inc. It is available for Microsoft Windows, Mac, and Linux OS. VMware also offers VMWare Player for free[2]. This software presents many features and capabilities in the virtual environment including the power of backup systems in snapshots. The following figure represents the version used in this lab (VMware Workstation Pro 16).

Figure 1: VMware Workstation Pro v16

Some computers may not start the VM machines or prompt out an error of VT-x is disable or does not enable to boot 64bit VM machines. That means that the virtualization is currently disabled in your BIOS.

[1]www.vmware.com/products/workstation-pro.html
[2]www.vmware.com/ products/player/

A.1.2 Setting Up Kali Linux

In this lab, we used a Debian-based Linux distribution called Kali Linux[3]. Kali Linux comes with many preinstalled tools for security and pentesting, including Metasploit Framework[4]. This project works under Kali Linux 2018.2

Figure 2: Kali Linux Distribution

[3]www.kali.org
[4]www.metasploit.com

A.2 Program listing

The source code of the APTML application can be found in this GitHub repository: https://github.com/GhnimiWael/APTML